TESTSIGMA — TAKE-HOME ASSIGNMENT

AI Engineer

ROLE	AI Engineer
DEPARTMENT	Engineering
TIME ESTIMATE	16 hours (2 focused days)
DEADLINE	5 days after receiving

Part 1 — Context

Testsigma is building the intelligence layer beneath agentic testing — a system that understands a running application semantically, connects it to product intent, and reasons about what's at risk when code changes. This assignment is the truest signal we know of for the work you'd actually do here: a working agent, built from scratch, that crawls a live application, ingests a product spec, builds a knowledge graph across three layers, and uses that graph to reason about real code changes.

This is not a coding test. It's a building test under realistic constraints — limited time, no scaffolding, your own choices on every dimension. We will read your code and your document with equal weight.

Part 2 — The Task

Two parts. The JD describes them in summary; this is the detailed brief. **AI assistants are explicitly permitted** — we care about judgment, not keystrokes. Code and design document are weighted equally.

Part A — The Working System

Build an agent that does all four of the following on a public web application of your choice (a real one — Amazon, GitHub, a public SaaS demo, anything with enough depth):

1. **Crawl.** A browser agent (browser-use, Playwright + LLM, your choice — your selection is part of the signal) explores the application autonomously and captures structured artifacts: DOM, screenshots, interaction transitions, screen relationships.
2. **Ingest.** A public PRD, README, product spec, GitHub wiki, or feature documentation you select. Parse it into a structured form your graph can use.
3. **Graph.** A Neo4j knowledge graph connecting three layers:

- **Requirements** — what was intended (from the PRD)
- **DOM / UI** — what was built (from the crawl)
- **Code** — how it was built (mapped from a public repository linked to the application)

The schema is yours to design. Justify your choices. Specifically: how do you model **absence** — the requirements that *should* be testable but aren't reflected anywhere in the captured UI?

4. **Reason.** Given a real Pull Request in the public repository, your system outputs the **blast radius**: which UI elements are at risk, which discovered user flows are affected, which requirements lose coverage. The output must be readable by a non-engineer who understands the product but not your code.

Hard scope note

16 hours is not enough time to do all four of the above to equal depth. That's deliberate. We want to see what you choose to invest depth in, and what you scope down — and we want you to *tell us* in the design document, not hide it. A submission that goes deep on two layers and explicitly scopes the third beats a submission that does all three shallowly.

A working narrow slice with honest scope decisions is the strong submission. Three half-done layers is the weak one.

Part B — The Design Document

Treat this as a document you would defend to engineers more senior than you. Be opinionated, show your trade-offs, name what you cut. Address each of the following at minimum:

5. **Agent decomposition —** what are the distinct stages of reasoning in your agent? Where are the boundaries? What is deterministic vs. LLM-driven, and why? We are not looking for a chain of prompts dressed up as an agent.
6. **Graph schema with justification —** your node and edge types, how they connect across the three layers, and specifically how you model absence. Justify your schema against a query you would need to run.
7. **Confidence handling under ambiguity —** when the agent isn't sure (the PRD describes a feature you can't find in the UI; the code change touches a function you can't map to a UI element), what happens? How is confidence represented? When does the system stop and ask a human?
8. **Eval approach —** how do you know your agent's output is right? If we ran your system 100 times on the same input, how would we know which runs were correct?
9. **Scope decisions —** what did you cut, what did you go deep on, and why? This is not optional. Submissions that present polished work without honest scope discussion will score lower on this dimension than submissions that admit and defend their cuts.
10. **What you would do with another week —** the three highest-value things you would build next, in order, with reasoning.

Part 3 — Deliverables

- **Code** in a GitHub repository, runnable, with a README we can follow end-to-end
- **Design document** — PDF or Markdown, roughly 8–12 pages, diagrams welcome
- **Sample output** — for a real PR you select, the blast-radius report your system produces, in the form a non-engineer QA lead would actually read
- **A 5–10 minute Loom** walking us through the prototype running and the document

Submission Format: GitHub repository URL + PDF/Markdown document + Loom URL

Submission Method: Email to [HM email] with subject line *[Candidate Name] — AI Engineer Assignment*